

CSC-3TC33

Algorithmique et fondements de l'informatique

Décidabilité et classes de complexité

Bertrand Meyer

Automne 2024

Décidabilité

La question de l'*Entscheidungsproblem*

~1928



David Hilbert
(1862-1943)

Entscheidungsproblem : Être capable de dire ce qu'un algorithme peut ou ne peut pas faire.

Nécessite un modèle de calcul.

Le calcul électromécanique dans les années 30



Programmation d'un IBM 405 « Alphanumeric Accounting Machine » de 1934
Précurseur des ordinateurs, permettant de calculer sur des cartes perforées.

Modèles de calcul principaux

Différents modèles de calcul ont été imaginés pour répondre à la question de l'*Entscheidungsproblem*.

Modèles de calcul principaux

Différents modèles de calcul ont été imaginés pour répondre à la question de l'*Entscheidungsproblem*.

- Le λ -calcul de Alonzo Church (~1930)
Basé sur le concept de fonction et d'application.
Fondement théorique de la programmation fonctionnelle.



Alonzo Church
(1903 †1995)

Modèles de calcul principaux

Différents modèles de calcul ont été imaginés pour répondre à la question de l'*Entscheidungsproblem*.

- Le λ -calcul de Alonzo Church (~1930)
Basé sur le concept de fonction et d'application.
Fondement théorique de la programmation fonctionnelle.
- Modèle de Herbrand-Gödel-Kleene
Notions de fonctions récursives



Alonzo Church
(1903 †1995)



Stephen C. Kleene
(1909 †1994)



Jacques Herbrand
(1908 †1931)

Modèles de calcul principaux

Différents modèles de calcul ont été imaginés pour répondre à la question de l'*Entscheidungsproblem*.

- Le λ -calcul de Alonzo Church (~1930)
Basé sur le concept de fonction et d'application.
Fondement théorique de la programmation fonctionnelle.
- Modèle de Herbrand-Gödel-Kleene
Notions de fonctions récursives
- Les machines de Alan Turing (~1939)
Basé sur les automates
Fondement théorique de l'architecture de von Neumann



Alonzo Church
(1903 †1995)



Stephen C. Kleene
(1909†1994)



Jacques Herbrand
(1908 †1931)



Alan Turing
(1912†1954)

Modèles de calcul principaux

Différents modèles de calcul ont été imaginés pour répondre à la question de l'*Entscheidungsproblem*.

- Le λ -calcul de Alonzo Church (~1930)
Basé sur le concept de fonction et d'application.
Fondement théorique de la programmation fonctionnelle.
- Modèle de Herbrand-Gödel-Kleene
Notions de fonctions récursives
- Les machines de Alan Turing (~1939)
Basé sur les automates
Fondement théorique de l'architecture de von Neumann



Alonzo Church
(1903 †1995)



Stephen C. Kleene
(1909†1994)



Jacques Herbrand
(1908 †1931)



Alan Turing
(1912†1954)

Thèse de Church : ces approches équivalentes sont la bonne manière de formaliser la notion de **méthode effective de calcul**.

- Dans ce cours, notre modèle de calcul est un programme python exécuté sur un ordinateur idéal (dont la mémoire est infinie).

- Dans ce cours, notre modèle de calcul est un programme python exécuté sur un ordinateur idéal (dont la mémoire est infinie).
- Plus généralement, on peut considérer comme modèle de calcul n'importe quel langage de programmation dit Turing complet

- Dans ce cours, notre modèle de calcul est un programme python exécuté sur un ordinateur idéal (dont la mémoire est infinie).
- Plus généralement, on peut considérer comme modèle de calcul n'importe quel langage de programmation dit Turing complet
- On supposera qu'il est possible de n'exécuter que les t premières opérations élémentaires d'un programme.

- Dans ce cours, notre modèle de calcul est un programme python exécuté sur un ordinateur idéal (dont la mémoire est infinie).
- Plus généralement, on peut considérer comme modèle de calcul n'importe quel langage de programmation dit Turing complet
- On supposera qu'il est possible de n'exécuter que les t premières opérations élémentaires d'un programme.
- Traditionnellement, on utilise surtout le modèle des machines de Turing.

Nous fixons un alphabet Σ . En pratique, il s'agit en général de l'alphabet ASCII.

Définition

On dit qu'une fonction $f : \Sigma^* \rightarrow \Sigma^*$, définie sur tout Σ^* , est **calculable** lorsqu'il existe un programme Python qui prend une chaîne de caractère w en entrée, termine toujours en temps fini, et admet $f(w)$ comme valeur de retour.

Ensemble décidable

Soient V et F deux mots de Σ^* .

Définition

On dit qu'un ensemble, ou langage, $A \subseteq \Sigma^*$ est **décidable** (ou calculable) lorsque sa fonction indicatrice $\mathbf{1}_A : \Sigma^* \rightarrow \{V, F\}$ est calculable.

Ensemble décidable

Soient V et F deux mots de Σ^* .

Définition

On dit qu'un ensemble, ou langage, $A \subseteq \Sigma^*$ est **décidable** (ou calculable) lorsque sa fonction indicatrice $\mathbf{1}_A : \Sigma^* \rightarrow \{V, F\}$ est calculable.

Autrement dit, il existe un algorithme qui prend en entrée $w \in \Sigma^*$, termine toujours en temps fini et renvoie « vrai » si $w \in A$ ou « faux » si $w \notin A$.

Les instance d'un problème de décision comme un langage

Définition

On appelle **problème de décision** toute question portant sur un énoncé mathématique dépendant d'une certaine entrée et dont la réponse est soit **oui** soit **non**.

Les instance d'un problème de décision comme un langage

Définition

On appelle **problème de décision** toute question portant sur un énoncé mathématique dépendant d'une certaine entrée et dont la réponse est soit **oui** soit **non**.

On parle d'**instance** d'un problème de décision pour désigner un jeu de données d'entrée et la spécialisation du problème à ce jeu de donnée.

Les instance d'un problème de décision comme un langage

Définition

On appelle **problème de décision** toute question portant sur un énoncé mathématique dépendant d'une certaine entrée et dont la réponse est soit **oui** soit **non**.

On parle d'**instance** d'un problème de décision pour désigner un jeu de données d'entrée et la spécialisation du problème à ce jeu de donnée.

Un problème de décision est **décidable** si le langage de ses instances est décidable, autrement dit, s'il existe un algorithme qui prend en entrée une instance I du problème, termine toujours en temps fini et renvoie « vrai » si l'instance I a pour réponse oui ou « faux » sinon.

Exemple

- Décider si un entier **est premier**.

Ensemble d'instances : entiers naturels $\mathbb{N}$.

- Décider si un graphe **est connexe**.

Ensemble d'instances : ensemble de tous les graphes.

Exemples

Exemple

L'ensemble des dénotations des nombres pairs, des carrés parfaits, des nombres premiers sont décidables.

Fonction calculable partielle

Définition

On dit qu'une fonction $f : \Sigma^* \rightarrow \Sigma^*$, définie partiellement sur Σ^* , est **calculable partielle** lorsqu'il existe un algorithme qui prend une chaîne de caractère w en entrée, termine en temps fini ssi $f(w)$ est définie, et renvoie $f(w)$ dans ce cas.

Fonction calculable partielle

Définition

On dit qu'une fonction $f : \Sigma^* \rightarrow \Sigma^*$, définie partiellement sur Σ^* , est **calculable partielle** lorsqu'il existe un algorithme qui prend une chaîne de caractère w en entrée, termine en temps fini ssi $f(w)$ est définie, et renvoie $f(w)$ dans ce cas.

On dit qu'un ensemble, ou un langage, $A \subseteq \Sigma^*$ est **semi-décidable** lorsque sa fonction partielle définie sur A et valant V partout est calculable partielle.

Fonction calculable partielle

Définition

On dit qu'une fonction $f : \Sigma^* \rightarrow \Sigma^*$, définie partiellement sur Σ^* , est **calculable partielle** lorsqu'il existe un algorithme qui prend une chaîne de caractère w en entrée, termine en temps fini ssi $f(w)$ est définie, et renvoie $f(w)$ dans ce cas.

On dit qu'un ensemble, ou un langage, $A \subseteq \Sigma^*$ est **semi-décidable** lorsque sa fonction partielle définie sur A et valant V partout est calculable partielle.

Autrement dit, il existe un algorithme qui prend en entrée $w \in \Sigma^*$, termine toujours en temps fini et renvoie « oui » si $w \in A$ et ne termine pas sinon.

Théorème

Un ensemble $A \subseteq \Sigma^$ est décidable ssi A et $\Sigma^* \setminus A$ sont semi-décidables.*

Décidable et semi-décidable

Théorème

Un ensemble $A \subseteq \Sigma^$ est décidable ssi A et $\Sigma^* \setminus A$ sont semi-décidables.*

Démonstration.

Si $A \subseteq \Sigma^*$ est décidable, on dispose d'un algorithme `decideA(w)` qui renvoie **True** ou **False** selon que w appartient à A ou non. On écrit

```
def semidecideA(w):  
    if decideA(w):  
        return True:  
    else:  
        while True:  
            pass
```

```
def semidecidenonA(w):  
    if not decideA(w):  
        return True:  
    else:  
        while True:  
            pass
```

qui semi-décident A et son complémentaire.

Décidable et semi-décidable

Théorème

Un ensemble $A \subseteq \Sigma^$ est décidable ssi A et $\Sigma^* \setminus A$ sont semi-décidables.*

Démonstration.

Si $A \subseteq \Sigma^*$ est décidable, on dispose d'un algorithme `decideA(w)` qui renvoie **True** ou **False** selon que w appartient à A ou non. On écrit

```
def semidecideA(w):  
    if decideA(w):  
        return True:  
    else:  
        while True:  
            pass
```

```
def semidecidenonA(w):  
    if not decideA(w):  
        return True:  
    else:  
        while True:  
            pass
```

qui semi-décident A et son complémentaire.

Si A et $\Sigma^* \setminus A$ sont semi-décidables, on dispose de deux algorithmes qui renvoient **True** si w appartient à A , respectivement w n'appartient pas à A et de ne se terminent pas sinon. On écrit un algorithme `decideA(w)` qui exécute en parallèle `semidecideA(w)` et `semidecidenonA(w)` et renvoie **True** dès que `semidecideA(w)` termine et **False** dès que `semidecidenonA(w)` termine. □

Théorème

Un ensemble $A \subseteq \Sigma^$ non vide est semi-décidable ss'il existe une fonction calculable $f : \mathbb{N} \rightarrow \Sigma^*$ telle que $f(\mathbb{N}) = A$.*

Décidable et semi-décidable

Théorème

Un ensemble $A \subseteq \Sigma^$ non vide est semi-décidable ss'il existe une fonction calculable $f : \mathbb{N} \rightarrow \Sigma^*$ telle que $f(\mathbb{N}) = A$.*

Démonstration.

Si A est semi décidable non vide, on dispose d'un algorithme **semidecideA(w)** qui renvoient **True** si w appartient à A et ne se terminent pas sinon et d'un mot w_0 . On considère alors le programme **g(w, t)** qui exécute **semidecideA(w)** pendant t étapes élémentaires. Si **semidecideA(w)** termine en moins de t étapes, **g(w, t)** renvoie w , sinon il renvoie l'élément w_0 . Il suffit alors de construire une bijection $\mathbb{N} \rightarrow \Sigma^* \times \mathbb{N}$ pour conclure.

Si'il existe une fonction calculable $f : \mathbb{N} \rightarrow \Sigma^*$ telle que $f(\Sigma^*) = A$, pour construire un algorithme **semidecideA(w)**, on calcule successivement $f(0), f(1), f(2)$, etc. Si l'on rencontre w , on s'arrête et renvoie **True**. □

Machine universelle et problème de l'arrêt

Énumération des fonctions calculables partielles

Les algorithmes sont représentables sur un alphabet fini : on peut donc énumérer toutes les fonctions calculables partielles $\Sigma^* \rightarrow \Sigma^*$:

$$\varphi_0, \varphi_1, \varphi_2, \varphi_3, \dots$$

De même, on peut énumérer tous les mots de Σ^* :

$$w_0, w_1, w_2, \dots$$

On parle de **codage de Gödel** : cela permet de raisonner seulement sur des ensembles d'entiers au lieu d'objets plus compliqués.



Kurt Gödel
(1906 †1978)

Machine universelle

Point crucial (admis)

Il existe une **machine universelle** : i.e. un algorithme U qui prend en entrée une chaîne de caractère e (codant un algorithme T) et une chaîne de caractère w et calcule la même chose que T sur l'entrée w .

Machine universelle

Point crucial (admis)

Il existe une **machine universelle** : i.e. un algorithme U qui prend en entrée une chaîne de caractère e (codant un algorithme T) et une chaîne de caractère w et calcule la même chose que T sur l'entrée w .

Informatiquement : dans un langage de programmation

Turing-complet, on peut écrire un *interpréteur* U pour le langage lui-même (ou pour un autre langage Turing-complet).

En python, il s'agit plus ou moins de la fonction **exec**.

Machine universelle

Point crucial (admis)

Il existe une **machine universelle** : i.e. un algorithme U qui prend en entrée une chaîne de caractère e (codant un algorithme T) et une chaîne de caractère w et calcule la même chose que T sur l'entrée w .

Informatiquement : dans un langage de programmation

Turing-complet, on peut écrire un *interpréteur* U pour le langage lui-même (ou pour un autre langage Turing-complet).

En python, il s'agit plus ou moins de la fonction **exec**.

Mathématiquement, on peut dire que la fonction (partielle)

$(e, n) \mapsto \varphi_e(w_n)$ (= résultat du e -ième algorithme appliqué sur la n -ième entrée) est elle-même calculable partielle.

Machine universelle

Point crucial (admis)

Il existe une **machine universelle** : i.e. un algorithme U qui prend en entrée une chaîne de caractère e (codant un algorithme T) et une chaîne de caractère w et calcule la même chose que T sur l'entrée w .

Informatiquement : dans un langage de programmation

Turing-complet, on peut écrire un *interpréteur* U pour le langage lui-même (ou pour un autre langage Turing-complet).

En python, il s'agit plus ou moins de la fonction **exec**.

Mathématiquement, on peut dire que la fonction (partielle)

$(e, n) \mapsto \varphi_e(w_n)$ (= résultat du e -ième algorithme appliqué sur la n -ième entrée) est elle-même calculable partielle.

Philosophiquement, la notion d'exécution d'un algorithme est elle-même algorithmique.

Problème de l'arrêt

Définition

On appelle **problème de l'arrêt** l'écriture d'une fonction Python `arret(prgm, arg)` prenant en entrée

- une chaîne de caractères `prgm` contenant le code source d'un programme python
- et une chaîne de caractères `arg` représentant des entrées pour le programme `prgm`

et qui, pour toutes chaînes `prgm` et `arg`, termine en un temps fini en renvoyant

- **True** si l'exécution de `prgm` sur `arg` termine en temps fini,
- et **False** sinon.

Problème de l'arrêt

Définition

On appelle **problème de l'arrêt** l'ensemble des couples $(e, n) \in \mathbb{N}^2$ tels que le e -ième algorithme termine sur la n -ième entrée.

$$\{(e, n) \in \mathbb{N}^2 : \varphi_e(w_n) \downarrow\}$$

(où la notation « $\varphi_e(w_n) \downarrow$ » signifie que $\varphi_e(w_n)$ est défini, i.e., l'algorithme termine).

Théorème (Turing)

Le problème de l'arrêt est **semi-décidable mais non décidable**.

Démonstration du problème de l'arrêt

Démonstration.

Supposons qu'il existe un programme python `arret(prgm, arg)` qui retourne **True** quand l'algorithme `prgm` s'arrête sur l'entrée `arg` et renvoie **False** sinon.

Démonstration du problème de l'arrêt

Démonstration.

Supposons qu'il existe un programme python `arret(prgm, arg)` qui retourne `True` quand l'algorithme `prgm` s'arrête sur l'entrée `arg` et renvoie `False` sinon.

Ecrivons alors

```
def pgm_contradictoire(x):  
    if arret(x, x):  
        while True:  
            pass  
    else:  
        return 42
```

Démonstration du problème de l'arrêt

Démonstration.

Supposons qu'il existe un programme python `arret(prgm, arg)` qui retourne **True** quand l'algorithme `prgm` s'arrête sur l'entrée `arg` et renvoie **False** sinon.

Ecrivons alors

```
def pgm_contradictoire(x):  
    if arret(x, x):  
        while True:  
            pass  
    else:  
        return 42
```

Mais alors, que fait `exec(pgm_contradictoire,pgm_contradictoire)`?

Démonstration du problème de l'arrêt

Démonstration.

Supposons qu'il existe un programme python `arret(prgm, arg)` qui retourne **True** quand l'algorithme `prgm` s'arrête sur l'entrée `arg` et renvoie **False** sinon.

Ecrivons alors

```
def pgm_contradictoire(x):  
    if arret(x, x):  
        while True:  
            pass  
    else:  
        return 42
```

Mais alors, que fait `exec(pgm_contradictoire,pgm_contradictoire)`?

- Si l'exécution du programme `pgm_contradictoire` sur l'entrée `pgm_contradictoire` se termine, alors, par examen du code, elle débouche sur une boucle infinie qui ne se termine pas.

Démonstration du problème de l'arrêt

Démonstration.

Supposons qu'il existe un programme python `arret(prgm, arg)` qui retourne **True** quand l'algorithme `prgm` s'arrête sur l'entrée `arg` et renvoie **False** sinon.

Ecrivons alors

```
def pgm_contradictoire(x):  
    if arret(x, x):  
        while True:  
            pass  
    else:  
        return 42
```

Mais alors, que fait `exec(pgm_contradictoire,pgm_contradictoire)`?

- Si l'exécution du programme `pgm_contradictoire` sur l'entrée `pgm_contradictoire` se termine, alors, par examen du code, elle débouche sur une boucle infinie qui ne se termine pas.
- Si l'exécution du programme `pgm_contradictoire` sur l'entrée `pgm_contradictoire` ne se termine pas, alors, par examen du code, elle se termine avec 42 comme valeur de retour.

Contradiction. Par conséquent, la fonction `arret(prgm, arg)` n'existe pas.



Démonstration du problème de l'arrêt

Corollaire

Le complémentaire du problème de l'arrêt n'est pas semi-décidable.

Démonstration du problème de l'arrêt

Corollaire

Le complémentaire du problème de l'arrêt n'est pas semi-décidable.

Démonstration.

Le problème de l'arrêt est semi-décidable. Si son complémentaire l'était aussi, alors le problème de l'arrêt serait décidable, ce qui n'est pas le cas. □

Généralisation : le théorème de Rice

Théorème (Rice)

Soit F un ensemble quelconque de fonctions partielles $\Sigma^* \rightarrow \Sigma^*$ et $A = \{e \in \mathbb{N}; \varphi_e \in F\}$. Si A est décidable, alors F est soit vide soit égale à l'ensemble de toutes les fonctions partielles $\Sigma^* \rightarrow \Sigma^*$.

Toute **propriété sémantique** non triviale d'un programme est **indécidable** : on ne peut rien déduire algorithmiquement par la seule lecture du code.

Conséquence

On ne peut pas décider par un algorithme

- si un programme rend la valeur zéro,
- si un programme termine sur une erreur d'exécution,
- si deux programmes calculent la même chose,
- si un programme calcule un résultat correct par rapport à sa spécification.

Classe de complexité

Définition

Étant donné un problème de décision et une manière de décrire une instance à l'aide d'un certain alphabet, on appelle **taille d'une instance** le nombre de symboles nécessaires pour décrire une instance.

Par habitude, on considère toujours un alphabet binaire.

Exemple

- Pour représenter un entier n , $\lceil \log_2 n \rceil$ bits sont nécessaires en base 2.

Exemple

- Pour représenter un entier n , $\lceil \log_2 n \rceil$ bits sont nécessaires en base 2.
- Pour représenter un graphe G à n sommets, on peut numéroter les sommets de G ; coder chaque sommet par $\lceil \log_2 n \rceil$ bits et présenter G sous la forme d'une suite d'arêtes, c-à-d une suite de couples de sommets. Le graphe se représente en moins de $2n^2 \lceil \log_2 n \rceil$ bits.

Définition

On dit qu'un algorithme est en **temps polynomial** si le nombre d'« opérations élémentaires » effectuées est polynomial en la taille de l'instance.

Définition

On dit qu'un algorithme est en **temps polynomial** si le nombre d'« opérations élémentaires » effectuées est polynomial en la taille de l'instance.

Définition

On appelle **P** la classe des problèmes de décision que l'on peut **décider en temps polynomial**, c'est-à-dire, tels qu'il existe un algorithme qui renvoie la réponse oui ou non en temps polynomial à partir d'une instance du problème.

Exemple

- Décider si un entier **est premier** appartient à **P**
- Décider si un graphe **est connexe** appartient à **P**.

Définition

On appelle **NP** la classe des problèmes de décision A que l'on peut vérifier en temps polynomial,

Définition

On appelle **NP** la classe des problèmes de décision A que l'on peut **vérifier en temps polynomial**, c'est-à-dire, tels qu'il existe un algorithme $\mathcal{A}$ qui prend en entrée une instance a du problème A et un certificat c et qui renvoie une réponse oui ou non (en temps polynomial en $|a|$) de sorte que

Définition

On appelle **NP** la classe des problèmes de décision A que l'on peut **vérifier en temps polynomial**, c'est-à-dire, tels qu'il existe un algorithme $\mathcal{A}$ qui prend en entrée une instance a du problème A et un certificat c et qui renvoie une réponse oui ou non (en temps polynomial en $|a|$) de sorte que

- si a admet la réponse oui au problème A , alors il existe au moins un certificat c_0 (de taille polynomiale en $|a|$) tel que $\mathcal{A}(a, c_0) = \text{oui}$.

Définition

On appelle **NP** la classe des problèmes de décision A que l'on peut **vérifier en temps polynomial**, c'est-à-dire, tels qu'il existe un algorithme $\mathcal{A}$ qui prend en entrée une instance a du problème A et un certificat c et qui renvoie une réponse oui ou non (en temps polynomial en $|a|$) de sorte que

- si a admet la réponse oui au problème A , alors il existe au moins un certificat c_0 (de taille polynomiale en $|a|$) tel que $\mathcal{A}(a, c_0) = \text{oui}$.
- si a admet la réponse non au problème A , alors pour tout c (de taille polynomiale en $|a|$), on a $\mathcal{A}(a, c) = \text{non}$.

Définition

On appelle **NP** la classe des problèmes de décision A que l'on peut **vérifier en temps polynomial**, c'est-à-dire, tels qu'il existe un algorithme $\mathcal{A}$ qui prend en entrée une instance a du problème A et un certificat c et qui renvoie une réponse oui ou non (en temps polynomial en $|a|$) de sorte que

- si a admet la réponse oui au problème A , alors il existe au moins un certificat c_0 (de taille polynomiale en $|a|$) tel que $\mathcal{A}(a, c_0) = \text{oui}$.
- si a admet la réponse non au problème A , alors pour tout c (de taille polynomiale en $|a|$), on a $\mathcal{A}(a, c) = \text{non}$.

Attention : on cherche uniquement à confirmer la réponse oui.

Exemple

GRAPHE HAMILTONIEN

Étant donné un graphe $G = (V, E)$, décider si le graphe G contient un cycle élémentaire C passant par chaque sommet une et une seule fois.

Exemple

GRAPHE HAMILTONIEN

Étant donné un graphe $G = (V, E)$, décider si le graphe G contient un cycle élémentaire C passant par chaque sommet une et une seule fois.

Proposition

GRAPHE HAMILTONIEN $\in$ NP

Exemple

GRAPHE HAMILTONIEN

Étant donné un graphe $G = (V, E)$, décider si le graphe G contient un cycle élémentaire C passant par chaque sommet une et une seule fois.

Proposition

GRAPHE HAMILTONIEN $\in$ NP

Démonstration.

On peut contrôler la réponse oui en fournissant l'instance G et le certificat candidat C où C est une suite d'arête C . L'algorithme consiste à vérifier que C passe par chaque sommet une et une seule fois, ce qu'on peut faire en temps polynomial. □

Inclusion $P \subseteq NP$

Proposition
On a l'inclusion

$$P \subseteq NP.$$

Inclusion $P \subseteq NP$

Démonstration.

Soit A un problème dans la classe P .

Inclusion $P \subseteq NP$

Démonstration.

Soit A un problème dans la classe P . Il existe donc un algorithme $\mathcal{A}_0$

Algorithme 3 : $\mathcal{A}_0$ (décision de A)

Entrées : Instance a de A

Sorties : Oui si l'instance a admet oui comme réponse au problème A ; Non sinon

Inclusion $P \subseteq NP$

Démonstration.

Soit A un problème dans la classe P . Il existe donc un algorithme $\mathcal{A}_0$

Algorithme 5 : $\mathcal{A}_0$ (décision de A)

Entrées : Instance a de A

Sorties : Oui si l'instance a admet oui comme réponse au problème A ; Non sinon

Considérons l'algorithme $\mathcal{A}$

Algorithme 6 : $\mathcal{A}$ (vérification de A)

Entrées : Instance a de A , certificat candidat c de a

Sorties : Oui si l'instance a admet oui comme réponse au problème A ; non sinon (quel que soit le certificat)

1 retourner $\mathcal{A}_0(a)$

Comparer la difficulté

Définition

Un problème de décision A est **polynomialement réductible** à un problème de décision B s'il existe un algorithme qui,

Définition

Un problème de décision A est **polynomialement réductible** à un problème de décision B s'il existe un algorithme qui,

- pour toute instance a du problème A , transforme a en une instance b du problème B en temps polynomial

Définition

Un problème de décision A est **polynomialement réductible** à un problème de décision B s'il existe un algorithme qui,

- pour toute instance a du problème A , transforme a en une instance b du problème B en temps polynomial
- tel que a admet la réponse oui au problème A si et seulement si b admet la réponse oui au problème B .

Définition

Un problème de décision A est **polynomialement réductible** à un problème de décision B s'il existe un algorithme qui,

- pour toute instance a du problème A , transforme a en une instance b du problème B en temps polynomial
- tel que a admet la réponse oui au problème A si et seulement si b admet la réponse oui au problème B .

On note dans ce cas $A \prec B$.

Interprétation : A est au plus aussi difficile que B .

Exemple

ENSEMBLE INDÉPENDANT

Étant donné un graphe $G = (V, E)$ et un entier k , existe-t-il un sous-ensemble de sommets $I \subseteq V$ de taille $\geq k$ tel que deux sommets quelconques de I ne sont jamais reliés ?

Exemple

ENSEMBLE INDÉPENDANT

Étant donné un graphe $G = (V, E)$ et un entier k , existe-t-il un sous-ensemble de sommets $I \subseteq V$ de taille $\geq k$ tel que deux sommets quelconques de I ne sont jamais reliés ?

CLIQUE

Étant donné un graphe $G = (V, E)$ et un entier k , existe-t-il un sous-ensemble de sommets $C \subseteq V$ de taille $\geq k$ tel que deux sommets quelconques de C sont toujours reliés ?

Exemple

ENSEMBLE INDÉPENDANT

Étant donné un graphe $G = (V, E)$ et un entier k , existe-t-il un sous-ensemble de sommets $I \subseteq V$ de taille $\geq k$ tel que deux sommets quelconques de I ne sont jamais reliés ?

CLIQUE

Étant donné un graphe $G = (V, E)$ et un entier k , existe-t-il un sous-ensemble de sommets $C \subseteq V$ de taille $\geq k$ tel que deux sommets quelconques de C sont toujours reliés ?

Exemple

On a

ENSEMBLE INDÉPENDANT $\prec$ CLIQUE

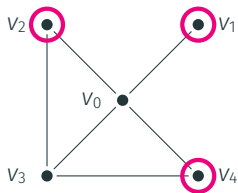
et

CLIQUE $\prec$ ENSEMBLE INDÉPENDANT.

Exemple : détail de la réduction polynomiale

Réduction polynomiale : construire le graphe complémentaire.

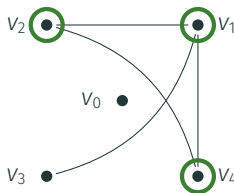
$$V = \{v_0, v_1, v_2, v_3, v_4\}$$



$\{v_1, v_2, v_4\}$ est indépendant

$$V' = \{v_0, v_1, v_2, v_3, v_4\}$$

$$E' = \bar{E}$$



$\{v_1, v_2, v_4\}$ est une clique

Exemple

ENSEMBLE INDÉPENDANT

Étant donné un graphe $G = (V, E)$ et un entier k , existe-t-il un sous-ensemble de sommets $I \subseteq V$ de taille $\geq k$ tel que deux sommets quelconques de I ne sont jamais reliés ?

Exemple

ENSEMBLE INDÉPENDANT

Étant donné un graphe $G = (V, E)$ et un entier k , existe-t-il un sous-ensemble de sommets $I \subseteq V$ de taille $\geq k$ tel que deux sommets quelconques de I ne sont jamais reliés ?

SOMMET-COUVERTURE

Étant donné un graphe $G = (V, E)$ et un entier k , existe-t-il un sous-ensemble de sommets $C \subseteq V$ de taille $\leq k$ tel que toute arête admet une extrémité dans C ?

Exemple

ENSEMBLE INDÉPENDANT

Étant donné un graphe $G = (V, E)$ et un entier k , existe-t-il un sous-ensemble de sommets $I \subseteq V$ de taille $\geq k$ tel que deux sommets quelconques de I ne sont jamais reliés ?

SOMMET-COUVERTURE

Étant donné un graphe $G = (V, E)$ et un entier k , existe-t-il un sous-ensemble de sommets $C \subseteq V$ de taille $\leq k$ tel que toute arête admet une extrémité dans C ?

Exemple

On a

ENSEMBLE INDÉPENDANT $\prec$ SOMMET-COUVERTURE

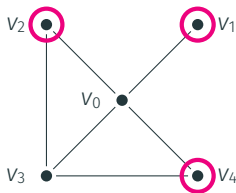
et

SOMMET-COUVERTURE $\prec$ ENSEMBLE INDÉPENDANT.

Exemple : détail de la réduction polynomiale

Réduction polynomiale : I est indépendant si et seulement si $C = V \setminus I$ est une couverture.

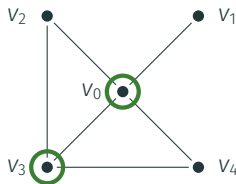
$$V = \{v_0, v_1, v_2, v_3, v_4\}$$



$\{v_1, v_2, v_4\}$ est indépendant

$$V' = \{v_0, v_1, v_2, v_3, v_4\}$$

$$E' = E$$



$\{v_0, v_3\}$ est une couverture

Les problèmes NP-complets

Définition

Un problème de décision A est **NP-difficile** si pour tout problème $\Pi \in \mathbf{NP}$, on a $\Pi \preceq A$.

Un problème de décision A est **NP-complet** s'il est **NP-difficile** et appartient à **NP**.

Les problèmes NP-complets

Définition

Un problème de décision A est **NP-difficile** si pour tout problème $\Pi \in \mathbf{NP}$, on a $\Pi \preceq A$.

Un problème de décision A est **NP-complet** s'il est **NP-difficile** et appartient à **NP**.

Les problèmes **NP-complet** sont les plus difficiles parmi les problèmes de la classe **NP**.

Les problèmes NP-complets

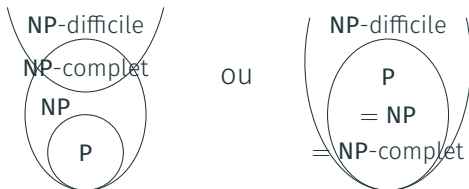
Définition

Un problème de décision A est **NP-difficile** si pour tout problème $\Pi \in \text{NP}$, on a $\Pi \preceq A$.

Un problème de décision A est **NP-complet** s'il est **NP-difficile** et appartient à **NP**.

Les problèmes **NP-complet** sont les plus difficiles parmi les problèmes de la classe **NP**.

La question à \$1.000.000



Des exemples de problèmes NP-complets

À partir de variables booléennes $x_1, x_2, \dots, x_n$, on définit :

- des **littéraux** $\lambda : x_1, \neg x_1, x_2, \neg x_2, \dots, x_n, \neg x_n$
- des **clauses** $C : \lambda_1 \vee \lambda_2 \vee \dots \vee \lambda_t$ (avec t littéraux distincts)
- des **formules** $\Phi : C_1 \wedge C_2 \wedge \dots \wedge C_s$.

À partir de variables booléennes $x_1, x_2, \dots, x_n$, on définit :

- des **littéraux** $\lambda : x_1, \neg x_1, x_2, \neg x_2, \dots, x_n, \neg x_n$
- des **clauses** $C : \lambda_1 \vee \lambda_2 \vee \dots \vee \lambda_t$ (avec t littéraux distincts)
- des **formules** $\Phi : C_1 \wedge C_2 \wedge \dots \wedge C_s$.

SAT : Étant donnée une formule Φ , existe-t-il un modèle, c-à-d une valuation des variables telle que l'interprétation de Φ soit la valeur de vérité « vrai » ?

Exemple

Exemple

La formule

$$\Phi = \underbrace{(x_1 \vee x_2 \vee \neg x_3)}_{C_1} \wedge \underbrace{(\neg x_1 \vee \neg x_2 \vee x_3)}_{C_2} \wedge \underbrace{(\neg x_1 \vee x_2 \vee x_3)}_{C_3} \wedge \underbrace{(x_1 \vee x_2 \vee x_4)}_{C_4}$$

Exemple

Exemple

La formule

$$\Phi = \underbrace{(x_1 \vee x_2 \vee \neg x_3)}_{C_1} \wedge \underbrace{(\neg x_1 \vee \neg x_2 \vee x_3)}_{C_2} \wedge \underbrace{(\neg x_1 \vee x_2 \vee x_3)}_{C_3} \wedge \underbrace{(x_1 \vee x_2 \vee x_4)}_{C_4}$$

est satisfiable.

Exemple

Exemple

La formule

$$\Phi = \underbrace{(x_1 \vee x_2 \vee \neg x_3)}_{C_1} \wedge \underbrace{(\neg x_1 \vee \neg x_2 \vee x_3)}_{C_2} \wedge \underbrace{(\neg x_1 \vee x_2 \vee x_3)}_{C_3} \wedge \underbrace{(x_1 \vee x_2 \vee x_4)}_{C_4}$$

est satisfiable.

On peut utiliser la valuation

x_1	V
x_2	F
x_3	V
x_4	V

comme modèle.

SAT est NP-complet

Théorème (Cook - Levin)

Le problème SAT est NP-complet.

~1971



Stephen Cook
(1939)



Leonid Levin
(1948)

SAT est NP-complet

Théorème (Cook - Levin)

Le problème SAT est NP-complet.

Esquisse de preuve.

Soit A un problème dans la classe **NP** et $\mathcal{A}(a, c)$ un algorithme qui vérifie le certificat c d'une instance a . Alors, on peut construire une formule de logique qui « simule » $\mathcal{A}$ et est satisfaite ss'il existe un certificat c tel que $\mathcal{A}(a, c)$ est vrai. On en déduit

$$A \preceq \text{SAT}.$$



~1971



Stephen Cook
(1939)



Leonid Levin
(1948)

SAT est NP-complet

Théorème (Cook - Levin)

Le problème SAT est NP-complet.

Esquisse de preuve.

Soit A un problème dans la classe **NP** et $\mathcal{A}(a, c)$ un algorithme qui vérifie le certificat c d'une instance a . Alors, on peut construire une formule de logique qui « simule » $\mathcal{A}$ et est satisfaite ss'il existe un certificat c tel que $\mathcal{A}(a, c)$ est vrai. On en déduit

$$A \preceq \text{SAT}.$$

Remarque

Tous les autres résultats de NP-complétude sont déduits du théorème de Cook-Levin.

~1971



Stephen Cook
(1939)



Leonid Levin
(1948)



ENSEMBLE INDÉPENDANT est NP-complet

ENSEMBLE INDÉPENDANT

Étant donné un graphe $G = (V, E)$ et un entier k , existe-t-il un sous-ensemble de sommets $I \subseteq V$ de taille $\geq k$ tel que deux sommets quelconques de I ne sont jamais reliés ?

ENSEMBLE INDÉPENDANT est NP-complet

ENSEMBLE INDÉPENDANT

Étant donné un graphe $G = (V, E)$ et un entier k , existe-t-il un sous-ensemble de sommets $I \subseteq V$ de taille $\geq k$ tel que deux sommets quelconques de I ne sont jamais reliés ?

Théorème

*Le problème ENSEMBLE INDÉPENDANT est **NP**-complet.*

ENSEMBLE INDÉPENDANT est NP-complet

ENSEMBLE INDÉPENDANT

Étant donné un graphe $G = (V, E)$ et un entier k , existe-t-il un sous-ensemble de sommets $I \subseteq V$ de taille $\geq k$ tel que deux sommets quelconques de I ne sont jamais reliés ?

Théorème

*Le problème ENSEMBLE INDÉPENDANT est **NP-complet**.*

Démonstration.

On va montrer

$3\text{-SAT} \preceq \text{ENSEMBLE INDÉPENDANT}$ et

Lemme

Le problème 3-SAT est **NP-complet**.



Réduction SAT $\prec$ 3-SAT

3-SAT : Étant donnée une formule F dont les clauses sont de longueur 3, existe-t-il un modèle, càd une valuation des variables telle que l'interprétation de F soit « vrai » ?

Lemme

Le problème 3-SAT est **NP**-complet.

Réduction SAT $\prec$ 3-SAT

3-SAT : Étant donnée une formule F dont les clauses sont de longueur 3, existe-t-il un modèle, c-à-d une valuation des variables telle que l'interprétation de F soit « vrai » ?

Lemme

Le problème 3-SAT est **NP**-complet.

Démonstration.

On remplace toute clause avec $t \geq 4$

$$C = (\lambda_1 \vee \lambda_2 \vee \cdots \vee \lambda_t)$$

par deux clauses

$$\Psi = (\lambda_1 \vee \cdots \vee \lambda_{t-2} \vee y) \wedge (\lambda_{t-1} \vee \lambda_t \vee \neg y)$$

où y est une variable supplémentaire. La clause C est satisfiable ssi Ψ l'est aussi.

Réduction $\text{SAT} \preceq 3\text{-SAT}$

3-SAT : Étant donnée une formule F dont les clauses sont de longueur 3, existe-t-il un modèle, c-à-d une valuation des variables telle que l'interprétation de F soit « vrai » ?

Lemme

Le problème 3-SAT est **NP**-complet.

Démonstration.

On remplace toute clause avec $t = 2$

$$C = (\lambda_1 \vee \lambda_2)$$

par

$$\psi = (\lambda_1 \vee \lambda_2 \vee y) \wedge (\lambda_1 \vee \lambda_2 \vee \neg y)$$

où y est une variable supplémentaire. La clause C est satisfiable ssi ψ l'est aussi.

Réduction 3-SAT $\prec$ ENSEMBLE INDÉPENDANT

Objectif : transformer une 3-formule en un problème de graphe.

Exemple

$$\Phi = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_4).$$

Réduction 3-SAT $\prec$ ENSEMBLE INDÉPENDANT

Objectif : transformer une 3-formule en un problème de graphe.

Définition

Un **gadget** est un morceau d'une instance qui simule le comportement d'un autre problème algorithmique.

Exemple

$$\Phi = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_4).$$

Réduction 3-SAT $\prec$ ENSEMBLE INDÉPENDANT

Objectif : transformer une 3-formule en un problème de graphe.

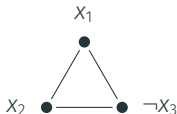
Définition

Un **gadget** est un morceau d'une instance qui simule le comportement d'un autre problème algorithmique.

Ici : on transforme chaque 3 clause en 1 triangle

Exemple

$$\Phi = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_4).$$



Réduction 3-SAT $\prec$ ENSEMBLE INDÉPENDANT

Objectif : transformer une 3-formule en un problème de graphe.

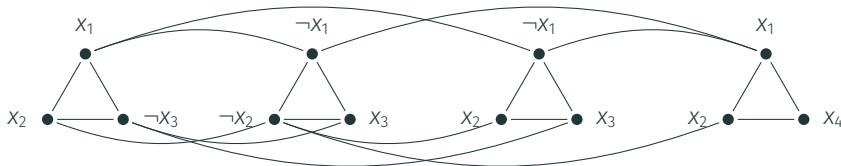
Définition

Un **gadget** est un morceau d'une instance qui simule le comportement d'un autre problème algorithmique.

Ici : on transforme chaque 3 clause en 1 triangle et on relie les littéraux opposés.

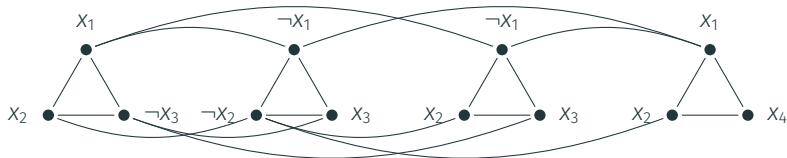
Exemple

$$\Phi = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_4).$$



Réduction 3-SAT $\prec$ ENSEMBLE INDÉPENDANT

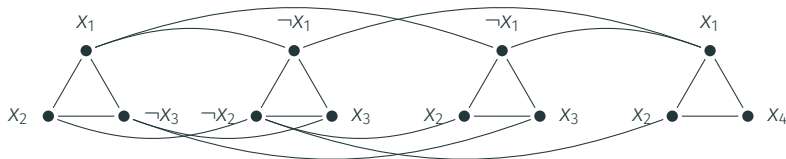
Exemple



$$\Phi = (X_1 \vee X_2 \vee \neg X_3) \wedge (\neg X_1 \vee \neg X_2 \vee X_3) \wedge (\neg X_1 \vee X_2 \vee X_3) \wedge (X_1 \vee X_2 \vee X_4)$$

Réduction 3-SAT $\prec$ ENSEMBLE INDÉPENDANT

Exemple



X_1	V
X_2	F
X_3	V
X_4	V

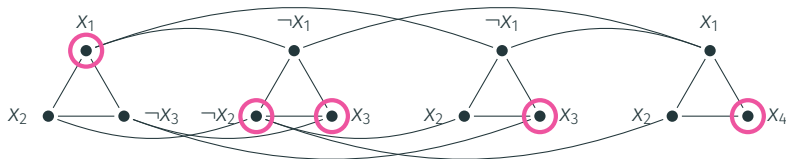
$$\Phi = (X_1 \vee X_2 \vee \neg X_3) \wedge (\neg X_1 \vee \neg X_2 \vee X_3) \wedge (\neg X_1 \vee X_2 \vee X_3) \wedge (X_1 \vee X_2 \vee X_4)$$

Fait 1

Une valuation avec $\geq t$ clauses valides donne un ensemble indépendant de plus de t sommets.

Réduction 3-SAT $\prec$ ENSEMBLE INDÉPENDANT

Exemple



X_1	V
X_2	F
X_3	V
X_4	V

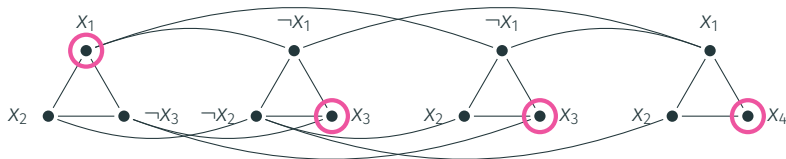
$$\Phi = (X_1 \vee X_2 \vee \neg X_3) \wedge (\neg X_1 \vee \neg X_2 \vee X_3) \wedge (\neg X_1 \vee X_2 \vee X_3) \wedge (X_1 \vee X_2 \vee X_4)$$

Fait 1

Une valuation avec $\geq t$ clauses valides donne un ensemble indépendant de plus de t sommets.

Réduction 3-SAT $\prec$ ENSEMBLE INDÉPENDANT

Exemple



X_1	V
X_2	F
X_3	V
X_4	V

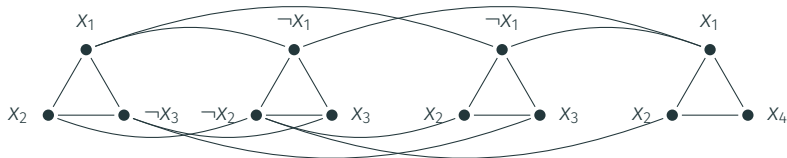
$$\Phi = (X_1 \vee X_2 \vee \neg X_3) \wedge (\neg X_1 \vee \neg X_2 \vee X_3) \wedge (\neg X_1 \vee X_2 \vee X_3) \wedge (X_1 \vee X_2 \vee X_4)$$

Fait 1

Une valuation avec $\geq t$ clauses valides donne un ensemble indépendant de plus de t sommets.

Réduction 3-SAT $\prec$ ENSEMBLE INDÉPENDANT

Exemple



$$\Phi = (X_1 \vee X_2 \vee \neg X_3) \wedge (\neg X_1 \vee \neg X_2 \vee X_3) \wedge (\neg X_1 \vee X_2 \vee X_3) \wedge (X_1 \vee X_2 \vee X_4)$$

Fait 1

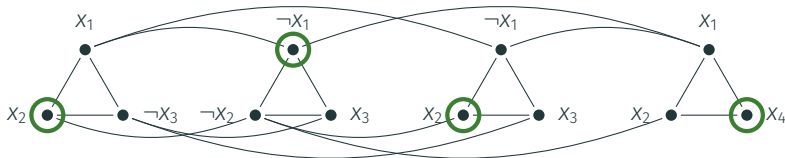
Une valuation avec $\geq t$ clauses valides donne un ensemble indépendant de plus de t sommets.

Fait 2

Un ensemble indépendant de plus de t sommets mène à un modèle de la formule.

Réduction 3-SAT $\prec$ ENSEMBLE INDÉPENDANT

Exemple



$$\Phi = (X_1 \vee X_2 \vee \neg X_3) \wedge (\neg X_1 \vee \neg X_2 \vee X_3) \wedge (\neg X_1 \vee X_2 \vee X_3) \wedge (X_1 \vee X_2 \vee X_4)$$

Fait 1

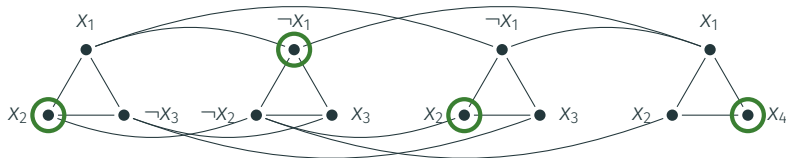
Une valuation avec $\geq t$ clauses valides donne un ensemble indépendant de plus de t sommets.

Fait 2

Un ensemble indépendant de plus de t sommets mène à un modèle de la formule.

Réduction 3-SAT $\prec$ ENSEMBLE INDÉPENDANT

Exemple



x_1	F
x_2	V
x_3	$?$
x_4	V

$$\Phi = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_4)$$

Fait 1

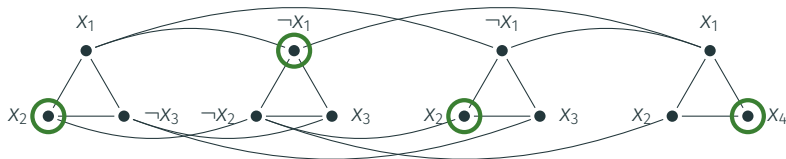
Une valuation avec $\geq t$ clauses valides donne un ensemble indépendant de plus de t sommets.

Fait 2

Un ensemble indépendant de plus de t sommets mène à un modèle de la formule.

Réduction 3-SAT $\prec$ ENSEMBLE INDÉPENDANT

Exemple



x_1	F
x_2	V
x_3	V
x_4	V

$$\Phi = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_4)$$

Fait 1

Une valuation avec $\geq t$ clauses valides donne un ensemble indépendant de plus de t sommets.

Fait 2

Un ensemble indépendant de plus de t sommets mène à un modèle de la formule.

Problèmes de décision vs problèmes d'optimisation

Définition

Un **problème d'optimisation** consiste à déterminer une solution réalisable d'une instance d'un problème qui maximise ou minimise une certaine fonction objectif.

Exemple de problème d'optimisation

PLUS COURT CHEMIN

Étant donné un graphe $G = (V, E)$ pondéré, deux sommets s et t de V , déterminer le plus court chemin entre s et t .

ARBRE COUVRANT DE POIDS MINIMUM

Étant donné un graphe $G = (V, E)$ pondéré, trouver un arbre couvrant dont le poids est minimum.

VOYAGEUR-EUSE DE COMMERCE

Étant donné un graphe $G = (V, E)$ pondéré, trouver un tour hamiltonien de coût minimum.

COUPE MAXIMUM

Étant donné un graphe $G = (V, E)$, trouver une coupe $(S, \bar{S})$ qui maximise le nombre d'arêtes coupées $|E(S, \bar{S})|$.

Transformation (cas de la minimisation)

Tout problème d'optimisation A_{opt} peut être transformé en un problème de décision A_d :

Problème de minimisation A_{opt}

Étant donné une instance a d'un problème A_{opt} , trouver x parmi les solutions réalisables de a qui minimise la fonction objectif $f_a(x)$.

Problème de décision A_d associé

Étant donné une instance a d'un problème A_{opt} et un seuil numérique k , existe-t-il une solution réalisable x tel que $f_a(x) \leq k$.

Transformation (cas de la maximisation)

Tout problème d'optimisation A_{opt} peut être transformé en un problème de décision A_d :

Problème de maximisation A_{opt}

Étant donné une instance a d'un problème A_{opt} , trouver x parmi les solutions réalisables de a qui maximise la fonction objectif $f_a(x)$.

Problème de décision A_d associé

Étant donné une instance a d'un problème A_{opt} et un seuil numérique k , existe-t-il une solution réalisable x tel que $f_a(x) \geq k$.

Exemple de problème d'optimisation transformés

PLUS COURT CHEMIN

Étant donnés un graphe $G = (V, E)$ pondéré, deux sommets s et t de V et un seuil ℓ , existe-t-il un chemin entre s et t de longueur $\leq \ell$?

ARBRE COUVRANT DE POIDS MINIMUM

Étant donnés un graphe $G = (V, E)$ pondéré et un seuil p , existe-t-il un arbre couvrant de poids $\leq p$?

VOYAGEUR-EUSE DE COMMERCE

Étant donné un graphe $G = (V, E)$ pondéré et un seuil c , existe-t-il un tour hamiltonien de coût $\leq c$?

COUPE MAXIMUM

Étant donné un graphe $G = (V, E)$ et un seuil k , existe-t-il une coupe $(S, \bar{S})$ qui coupe plus de k arêtes?